Turnitout

Similarity report

Public-web overlap only. Not a Turnitin report.

Generated 2026-10-31 16:07:45 UTC

12%

Overall similarity

No passage of 7 or more words matched a public page that this check could open.

Matched words 0 of 4,516
Sources 0
Longest match 0 words
Coverage searched 54 of 151 segments

Searched 54 of 151 text segments with 54 search phrases.

97 segments were not searched (limit of 54 phrases per check). Text in those parts could still match pages that were not looked up, so the real similarity can be higher than shown.

DuckDuckGo was switched off for this check (DuckDuckGo showed a bot challenge). Other engines were used.

22 candidate pages could not be read (HTTP 403 ×11, No readable text on the page ×6, UND_ERR_CONNECT_TIMEOUT ×2), so they could not be compared.

Stopped opening new pages after 80 to stay polite to other sites.

21 words in quotations or citations were excluded from the comparison and from the percentage.

Quotations and citation markers were excluded before comparison.

Sources

No public page produced a word run long enough to count as a match.

Document

LumiPath

Adaptive Pedestrian-Safety Streetlight System

Project Report

Course Name: Design of Smart City

Course Code: BCSE316L

Team Members

Rohan Mishra (23BCT0130)

Ekant Aman (23BCT0180)

Anurag Pareek (23BCT0173)

LumiPath | BCSE316L Design of Smart City | Page 1

Abstract

Streetlights in most Indian towns and cities stay on all night, whether or not anyone is on the road. This wastes electricity, and at the same time many roads are still poorly lit at the exact spot where a pedestrian is crossing. In this project we built LumiPath, a small working model of a streetlight that thinks before it lights up.

LumiPath uses an Arduino Uno, a light sensor (LDR), two infrared (IR) sensors, an HC-SR04 ultrasonic sensor, three LEDs (green, yellow and red), a buzzer and a push button. The light stays completely off in the daytime. When it gets dark, the system waits for a person or object to come close. As soon as motion is detected, all three LEDs switch on, and the moment the person leaves, they switch off again. The ultrasonic sensor keeps measuring the distance and speed of the approaching object, and a push button works as an SOS switch that turns the buzzer on and off. We also built a simple dashboard that shows the live sensor values and the state of the lamp on a laptop.

The prototype worked as planned in our tests. It shows how a low-cost, simple circuit can cut down energy waste and make roads safer for people walking at night, which fits well into the idea of a smart city. The report also maps the project to the Smart Cities Mission themes and to the UN Sustainable Development Goals 3, 7, 9, 11 and 13, with estimated energy and carbon metrics.

1. Introduction

A smart city is not only about big technology. Often it is about small, sensible improvements to things we see every day, and streetlights are a good example. They are everywhere, they run every night, and they use a lot of power across a whole city.

A normal streetlight works on a fixed timer or a simple dusk-to-dawn switch. It does not know if a person is walking under it. So it burns at full brightness on an empty road at 3 AM, and it may still be too dim or badly placed for someone crossing at 9 PM.

The idea behind LumiPath is to make the streetlight respond to the situation:

- If it is daytime, do nothing.
- If it is night and the road is empty, stay off to save energy.
- If it is night and someone is there, light up right away.
- If someone is in trouble, let them raise an alarm with a button.

We built this as a hardware prototype on a breadboard so that we could actually see and test each behaviour, instead of only describing it on paper.

2. Problem Statement

Conventional streetlights waste energy by staying on when nobody needs them, and they do not react to pedestrians in any way. There is also no easy way for a person walking alone at night to call for attention. We need a low-cost system that switches on only when it is dark and someone is present, that can measure how close and how fast an object is approaching, and that has a basic emergency alert.

LumiPath | BCSE316L Design of Smart City | Page 2

3. Objectives

1. To keep the lamp off during the day and turn it on only at night.
2. To detect a pedestrian or object using IR sensors and switch the lights on instantly.
3. To switch the lights off immediately when the motion is gone, so no power is wasted.
4. To measure the distance and approximate speed of an object using an ultrasonic sensor.
5. To provide a push-button SOS buzzer that toggles on and off.
6. To show all live readings on a simple dashboard for easy monitoring.

4. Scope of the Project

This project is a small prototype. It uses LEDs in place of a real street lamp and a breadboard in place of a proper PCB. The logic, however, is the same logic that would run on a real streetlight controller. With stronger drivers (relays or MOSFETs) and real lamps, the same design could be used on an actual pole.

5. Smart City Umbrella and Relevance

LumiPath connects to several smart city ideas that we studied in this course:

- Energy efficiency: Lights run only when needed, which reduces electricity use and cost for the

municipality.

- Public safety: Pedestrians get light exactly where and when they are present, and the SOS button gives a way to call attention.
- Use of IoT and sensors: The system senses its environment and takes decisions on its own, without a person operating it.
- Data for planning: The distance and speed readings could later be sent to a city server to understand footfall and traffic patterns.
- Low cost and scalable: All the parts are cheap and easily available, so the same design can be copied across many poles.

The table below shows how LumiPath fits into the main themes (the "umbrella") of the Smart Cities Mission, which covers both area-based development and pan-city solutions. Smart street lighting is a typical pan-city solution because the same unit can be repeated on every pole.

Smart city theme

What LumiPath does

Where it shows in the project

Smart energy

Keeps the lamp off unless it is dark

and someone is present

LDR + IR logic, instant switch-off

Smart safety and

security

Lights up for pedestrians and gives an

SOS buzzer

IR sensors, push button, buzzer

Smart infrastructure

Turns an ordinary pole into a sensing

pole

Arduino controller with 5 inputs and 4

outputs

IoT and ICT

Senses, decides and reports readings

by itself

Serial data feeding the dashboard

Smart mobility

Measures distance and speed of an

approaching object

HC-SR04 ultrasonic sensor

LumiPath | BCSE316L Design of Smart City | Page 3

LumiPath | BCSE316L Design of Smart City | Page 4

Smart city theme What LumiPath does Where it shows in the project

Smart environment Cuts electricity use and related carbon

emissions

Energy and CO2 estimates in Section

16

6. SDG Mapping and Metrics

LumiPath supports the following UN Sustainable Development Goals. The metrics in the last column are the numbers we use to judge the contribution. The energy and CO2 values are estimates worked out in Section 16, not field measurements. SDG Target How LumiPath contributes Metric SDG 7:

Affordable and

Clean Energy

7.3 Energy

efficiency

Lamp runs only when needed at

night

kWh saved per pole per year;

estimated 49% to 89% saving

depending on how busy the

road is

SDG 9:

Industry,

Innovation

and

Infrastructure

9.1 Reliable and

resilient

infrastructure

Low-cost automation added to

existing poles

Number of sensors used (4

types); response time of about

0.2 s per loop

SDG 11:

Sustainable

Cities and

Communities 11.7 Safe public

spaces Light on demand and

SOS alert for people walking

alone

Time from detection to light

ON; SOS available at any time

SDG 3: Good Health and

Well-being 3.6

Road traffic

injuries

Better visibility at the spot

where a pedestrian is present

Share of night hours with a

pedestrian lit (aim: 100% of

detected presence) SDG 13:

Climate Action

13.2 Climate

measures in

planning

Less electricity means less

emission from power plants

kg CO2 avoided per pole per

year; about 1,945 tonnes per

year for 10,000 poles at 20%

presence

7. Methodology Outline

We followed a simple build-and-test method, shown in Figure 1. Each step is explained below.

1. Problem identification: We studied how normal streetlights waste power and ignore pedestrians.
2. Requirement analysis: We listed what the system must do, such as day/night sensing, instant response, distance and speed reading, and an SOS option.

3. Component selection: We picked cheap and easily available parts, mainly an Arduino Uno, LDR, IR sensors, an HC-SR04, LEDs, a buzzer and a push button.
4. Circuit design: We decided the pin for each part and wired everything on a breadboard.
5. Arduino programming: We wrote the code in stages, first the sensors, then the LED logic, then the SOS button and the speed calculation.
6. Dashboard development: We built a web dashboard to show the live readings and the lamp state.
7. Testing and calibration: We covered the LDR, moved a hand near the sensors and pressed the button. We tuned the thresholds to stop flicker. Wrong behaviour sent us back to the programming step.
8. Analysis and reporting: We compared the system against the objectives, estimated the energy savings and prepared this report.

Figure 1: Methodology followed for the LumiPath project

8. Module-wise Activities

Module

The table below connects the course modules to the work we did in this project. Module names are grouped by theme.

Theme

Module 1

Introduction to smart cities

and the Smart Cities Mission

Activity done in LumiPath

Studied the smart city themes, chose smart street

lighting as the problem and mapped it to the Mission

themes and SDGs (Sections 5 and 6).

Module 2

Smart infrastructure and

energy

Designed an energy-saving streetlight that stays off

in daylight and on empty roads, and estimated the

energy saved per pole (Section 16).

Module 3

Sensing, IoT and controllers

Selected and wired the LDR, two IR sensors, the HC

SR04 and the push button to an Arduino Uno; wrote

the control code and fixed the day/night flicker with

two thresholds (Sections 10 to 13).

Module 4

Data, dashboards and

analytics

Printed live sensor values on the serial monitor, built

the web dashboard, calculated distance and speed,

and plotted the graphs (Sections 14 and 16).

Module 5

Safety, mobility and

sustainability

Added the SOS buzzer for personal safety, tested

pedestrian detection at night and linked the results

to SDGs 3, 7, 9, 11 and 13 (Sections 6 and 15).

9. Components Used

Component

Quantity

Purpose

Arduino Uno

1

Main controller that reads sensors and controls outputs

LumiPath | BCSE316L Design of Smart City | Page 5

Component

Quantity

Purpose

LDR (light sensor module)

1

Tells whether it is day or night

IR obstacle sensor modules

2

Detect a person or object nearby

HC-SR04 ultrasonic sensor

1

Measures distance, used to find speed

LEDs (green, yellow, red)

3

Act as the streetlight lamp

Buzzer

1

SOS alarm

Push button

1

Manual SOS switch

Resistors

As

needed

Protect the LEDs

Breadboard and jumper wires -

For making connections

USB cable and laptop

1

Power, programming and serial monitor

10. System Design

10.1 Block Diagram (in words)

The system has three parts: inputs, the controller, and outputs.

- Inputs: LDR, IR sensor 1, IR sensor 2, ultrasonic sensor, SOS push button.
- Controller: Arduino Uno, which runs our program in a continuous loop.
- Outputs: Green, yellow and red LEDs, the buzzer, and the serial monitor (which feeds the dashboard on the laptop).

The flow is simple: sensors send data to the Arduino, the Arduino decides what to do, and then it switches the LEDs and buzzer.

10.2 Pin Connections

Device

Arduino Pin

Green LED

D13

Yellow LED

D12

Red LED

D4

IR Sensor 1

D5

IR Sensor 2

D6

Ultrasonic Trigger

D9

Ultrasonic Echo

D10

LDR

A0

Buzzer

D8

SOS Push Button

D7 (with internal pull-up)

LumiPath | BCSE316L Design of Smart City | Page 6

10.3 Circuit Photographs

Figure 2 shows our complete breadboard setup. The Arduino Uno sits at the top and is connected by USB to the laptop. The two IR sensor modules and the HC-SR04 ultrasonic sensor are placed on the breadboard, along with the green, yellow and red LEDs.

Figure 2: Complete LumiPath circuit on the breadboard with Arduino Uno

Figure 3 shows the circuit while it is being tested. The Arduino is powered on, and the yellow LEDs light up as the hand is brought close to the IR sensors.

Figure 3: Circuit under test with a hand placed near the IR sensors

11. Working Principle

LumiPath | BCSE316L Design of Smart City | Page 7

11.1 Day and Night Detection

The LDR is connected to the analog pin A0. The Arduino reads a value between 0 and 1023. If the value goes above 900, the system decides that it is night. It goes back to day mode only when the value falls below 850.

We used two different numbers on purpose. If we had used just one number, the reading would keep jumping around that value and the system would flicker between day and night. The gap between 850 and 900 (called hysteresis) keeps the mode steady.

There is one more small trick. The LEDs themselves give out light, and this light can fall on the LDR and make the system think it is daytime. To avoid this, the program updates the day/night mode only when no motion is detected, that is, when the LEDs are off.

Figure 4 illustrates this idea. The top graph shows a light reading slowly getting darker, with the two thresholds marked. The bottom graph compares a single-threshold design, which flips back and forth, with the LumiPath design, which changes mode only once.

Figure 4: Day/night switching with two thresholds (illustrative signal)

11.2 Motion Detection

The two IR sensors give a LOW signal when something comes in front of them. The program treats it as motion if either one of them is LOW. Using two sensors gives a wider area of coverage on the road.

11.3 LED Control

- Day mode: All LEDs stay OFF, even if there is motion.
- Night mode with motion: Green, yellow and red LEDs turn ON together instantly.
- Night mode without motion: All LEDs turn OFF immediately.

There is no fade or delay on purpose, because the goal was an instant response and zero wasted power.

11.4 Distance and Speed Measurement

The HC-SR04 sends a short ultrasonic pulse and listens for the echo. The time taken by the echo is converted into distance using the speed of sound: distance (cm) = (echo time in microseconds × 0.034) / 2

The program remembers the previous distance and the previous time. Speed is then found as the change in distance divided by the time between the two readings: speed (cm/s) = |current distance ? previous distance| / time gap

These values are printed on the serial monitor and shown on the dashboard.

11.5 SOS Button

The push button uses the Arduino's internal pull-up resistor, so the pin reads HIGH normally and LOW when pressed. The program detects the moment the button goes from HIGH to LOW and flips the buzzer state:

- First press: buzzer ON
- Second press: buzzer OFF

A short 200 ms delay after each press avoids false double-presses caused by button bounce. The SOS works independently of the day/night mode, so it can be used at any time.

12. Algorithm

1. Start and set up all pins. Begin serial communication at 9600 baud.
2. Read the LDR value from A0.
3. Read both IR sensors and decide if motion is present.
4. If there is no motion, update day/night mode using the 900 and 850 thresholds.
5. Read the push button. If it was just pressed, flip the buzzer state. Apply this state to the buzzer.
6. Trigger the ultrasonic sensor, measure the echo and calculate distance.
7. Calculate speed from the previous distance and time.
8. Print LDR value, IR states, distance and speed on the serial monitor.
9. If it is night and motion is present, turn all LEDs ON. Otherwise turn all LEDs OFF.
10. Wait 200 ms and go back to step 2.

13. Software Implementation

The program is written in Arduino C++ in the Arduino IDE. It is organised into three simple parts:

- Pin and variable declarations: All pin numbers, thresholds and state variables are defined at the top, so they can be changed easily.
- setup(): Sets pins as input or output and starts the serial monitor.
- loop(): Runs again and again. It reads sensors, handles the button, calculates distance and speed, prints the readings and controls the LEDs.

We also wrote a small helper function, controlAllLEDs(), which switches all three LEDs together. This keeps the main loop clean and easy to read.

14. Dashboard (Simulator)

LumiPath | BCSE316L Design of Smart City | Page 9

Along with the hardware, we built an interactive web dashboard called the Adaptive Streetlight Interactive Visualizer. It shows:

- A drawing of a road with a streetlight pole and a light cone that appears when the lamp is on.
- The ambient light reading from the LDR as a bar, with the night and day threshold marks.
- The state of both IR sensors.
- The ultrasonic distance (in cm) and the object speed (in cm/s).
- The status of the three LED arms (green, yellow, red).

In our demonstration, the dashboard was running on the laptop right next to the circuit, as seen in Figure 5. It showed readings such as a distance of around 30 cm and a speed of a few cm/s while the lamp was active. This made it much easier to explain what the sensors were doing at any moment.

Figure 5: Dashboard running on the laptop beside the hardware circuit

15. Results and Testing

We tested the circuit by changing the light level and moving a hand in front of the sensors.

Test Condition	Expected Behaviour	Result
Bright light on the LDR, hand near IR sensor	LEDs stay OFF	Passed
Dark (LDR covered), no motion	LEDs stay OFF	
Dark, hand placed near IR sensor		

Passed

Green, yellow and red LEDs turn ON

immediately

Dark, hand removed

LEDs turn OFF immediately

Passed

Hand moved towards ultrasonic sensor

Passed

Distance falls and speed value changes on

the serial monitor

SOS button pressed once

Buzzer turns ON

Passed

SOS button pressed again

Passed

Buzzer turns OFF

Passed

In Figure 6, the green, yellow and red LEDs can be seen glowing while a hand is held over the IR sensors, which is exactly how the system is supposed to behave at night. LumiPath | BCSE316L Design of Smart City | Page 10

Figure 6: LEDs glowing when motion is detected by the IR sensors

16. Performance Metrics and Impact Analysis

16.1 Prototype Metrics

Metric

Value

Source

Day/night thresholds

900 (night), 850 (day)

Hysteresis band

Arduino code

50 ADC counts (about

4.9% of 1023)

Loop period

200 ms

Arduino code

Sensors used

Arduino code LDR, 2

IR, ultrasonic, push

button Outputs

Hardware 3 LEDs, 1

buzzer, serial data

Serial speed

9600 baud

Hardware

LED response to motion

Arduino code

Within one loop cycle

(about 0.2 s)

Demo reading shown on dashboard

Observed in testing

About 30 cm distance, a

few cm/s speed

16.2 Estimated Energy and Carbon Savings Dashboard screenshot

We could not measure a real lamp on our breadboard, so we made an estimate using clear assumptions.

These values are for explaining the idea and should not be read as measured results.

- Lamp power: 70 W per pole (a typical LED street lamp).
- Night length: 12 hours per night, 365 nights a year.
- Conventional lamp: on for the whole night, so 70 W x 12 h x 365 = about 307 kWh per year.
- LumiPath: lamp is on only while someone is present. We tried four cases, 10%, 20%, 30% and 50% of the night.
- Controller: the Arduino itself uses about 0.5 W all the time, which adds around 2 kWh per year.
- Emission factor: 0.8 kg of CO2 per kWh of grid electricity (assumed).

LumiPath | BCSE316L Design of Smart City | Page 11

Someone present

LumiPath

(kWh/yr)

Saving per pole

(kWh/yr)

Saving (%)

10% of night

33

274

CO2 avoided, 10,000

poles (tonnes/yr)

89%

20% of night

2,190

64

243

79%

30% of night

1,945

94

212

69%

50% of night

1,699

155

151

49%

1,209

Figure 7 shows the same numbers as a graph. The saving is highest on quiet roads, which is exactly where conventional lights waste the most power. Even on a busy road with someone present half of the night, the estimated saving is close to half.

Figure 7: Estimated yearly energy per pole, conventional versus LumiPath

These figures ignore dimming and the extra cost of real lamp drivers, so the real saving will differ. A field trial with a power meter would be the next step to confirm them.

17. Advantages

- Saves electricity because the light is on only when it is needed.
- Quick response, with no noticeable delay in switching.
- Low cost and made from easily available parts.
- No flickering between day and night, thanks to the two-threshold design.
- Gives extra safety with the SOS buzzer.
- Easy to extend with more sensors or a wireless module.

18. Limitations

- The prototype uses small LEDs. A real street lamp needs a relay or a driver circuit.
- IR sensors have a short range and can be affected by strong sunlight.
- The speed value is only displayed. It is not yet used to take any decision, such as making the light brighter for a fast-moving vehicle.
- The ultrasonic reading can be a little noisy, which affects the speed value.

LumiPath | BCSE316L Design of Smart City | Page 12

- The loop has a 200 ms delay, so very quick movements may be missed for a moment.
- The LDR thresholds depend on the surroundings and may need adjusting at a new location.
- The buzzer is local only. It does not send an alert to anyone.

19. Future Scope

- Add a Wi-Fi module such as ESP8266 or ESP32 so that data goes to a cloud server and many poles can be monitored from one place.
- Use PWM to dim the lamp in steps instead of a plain ON/OFF, for example a dim glow on an empty road and full brightness when someone arrives.
- Use the speed reading to tell a pedestrian from a vehicle and change the lighting accordingly.
- Send the SOS signal as an SMS or app notification to the police or a trusted contact.
- Add solar power with a battery so the pole works without the grid.
- Add a camera or a better sensor such as PIR or radar for longer range detection.
- Use stored data to study footfall and traffic and help city planners.

20. Conclusion

LumiPath shows that a smart streetlight does not need to be complicated or expensive. With just an Arduino and a handful of cheap sensors, we were able to make a light that stays off in daylight, wakes up only when it is dark and someone is present, switches off the moment the person leaves, measures distance and speed, and raises an alarm on request. The hardware worked reliably in our tests, and the dashboard made the behaviour easy to see and explain.

Through this project we learned how sensors, a microcontroller and simple logic can be combined to solve a real city problem. We also learned practical things such as avoiding flicker with two thresholds and handling button bounce. We believe that a scaled-up version of this idea can save energy and make night-time roads safer in a real smart city.

21. References

1. Arduino official documentation, Arduino Uno Rev3 and Arduino language reference, arduino.cc.

2. HC-SR04 ultrasonic ranging module datasheet.

3. Documentation for IR obstacle avoidance sensor modules and LDR light sensor modules.

4. Course material, Design of Smart City (BCSE316L), VIT Vellore.

5. Articles and reports on smart street lighting and energy-efficient public lighting systems.

Appendix: Arduino Code

// LumiPath: Adaptive Pedestrian-Safety Streetlight System

// Final Code: Instant LED Turn-OFF on Motion Removal + Strict Night Mode + Toggle SOS + Ultrasonic Speed

// LED Pins Configuration

const int greenLED = 13;

// Green LED pin const

int yellowLED = 12; //

Yellow LED pin const int

redLED = 4;

// Red LED pin

LumiPath | BCSE316L Design of Smart City | Page 13

LumiPath | BCSE316L Design of Smart City | Page 14

// Sensor and Control Pins Configuration const int

irSensor1 = 5; // First IR Sensor pin const int

irSensor2 = 6; // Second IR Sensor pin const int

trigPin = 9; // Ultrasonic Trigger pin const int

echoPin = 10; // Ultrasonic Echo pin const int ldrPin

= A0; // LDR Light Sensor analog pin const int

buzzerPin = 8; // Buzzer pin const int sosButtonPin

= 7; // Push Button SOS pin

```
// Safe LDR Thresholds with wide gap to completely stop Day/Night flickering
const int nightThreshold = 900; // Enter Night Mode stably above 900 const
int dayThreshold = 850; // Return to Daytime Mode only below 850 bool
isNightMode = false; // State tracker for Night/Day

// Variables for Object Speed Calculation
float previousDistance = 0; unsigned
long previousTime = 0;

// Variables for Push Button Toggle Logic (Press 1 = ON, Press 2 = OFF)
bool buzzerState = false; int lastButtonState = HIGH;

void setup() {
// Initialize Serial Monitor for debugging
Serial.begin(9600);

// Configure LED and Buzzer pins as OUTPUT
pinMode(greenLED, OUTPUT);
pinMode(yellowLED, OUTPUT);
pinMode(redLED, OUTPUT);
pinMode(buzzerPin, OUTPUT);

// Configure Sensor pins
pinMode(irSensor1, INPUT);
pinMode(irSensor2, INPUT);
pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);
pinMode(ldrPin, INPUT);
```

```
// Configure Push Button pin as INPUT with internal pull-up resistor
pinMode(sosButtonPin, INPUT_PULLUP);
}
void loop() {
// 1. Read LDR Value to determine Day/Night mode securely
int ldrValue = analogRead(ldrPin);

// Read IR Sensors for motion detection first to check if motion is currently active
int ir1State = digitalRead(irSensor1); int ir2State = digitalRead(irSensor2); bool
motionDetected = (ir1State == LOW || ir2State == LOW);

// Only update Day/Night mode when motion LEDs are NOT active, //
preventing the LEDs' own light from triggering a false Daytime switch.
if (!motionDetected) { if
(ldrValue > nightThreshold) {
isNightMode = true;
} else if (ldrValue < dayThreshold) {
isNightMode = false;
}
}

// 2. Push Button Toggle Logic for Buzzer Control (Independent)
int currentButtonState = digitalRead(sosButtonPin);
```



```
(currentButtonState == LOW && lastButtonState == HIGH) {
buzzerState = !buzzerState; if (buzzerState) {
```

```
Serial.println(">>> SOS EMERGENCY: Buzzer turned ON manually! <<<");
} else {
Serial.println(">>> SOS EMERGENCY: Buzzer turned OFF manually! <<<");
}
delay(200);
}
lastButtonState = currentButtonState;

// Apply the toggle state to the buzzer pin continuously
digitalWrite(buzzerPin, buzzerState ? HIGH : LOW);

// 4. Ultrasonic Sensor Distance and Speed Calculation
digitalWrite(trigPin, LOW); delayMicroseconds(2);
digitalWrite(trigPin, HIGH); delayMicroseconds(10);
digitalWrite(trigPin, LOW);

unsigned long currentTime = millis(); long duration = pulseIn(echoPin,
HIGH); float currentDistance = duration * 0.034 / 2.0; // Calculate
distance in cm

// Calculate object speed float timeInterval = (currentTime
- previousTime) / 1000.0; float speed = 0.0;

if (timeInterval > 0) { speed = abs(currentDistance -
previousDistance) / timeInterval;
}

previousDistance = currentDistance;
```

```
previousTime = currentTime;


// Print live metrics cleanly to the Serial Monitor
Serial.print("LDR: ");
Serial.print(ldrValue);
Serial.print(" | IR1: ");
Serial.print(ir1State);
Serial.print(" | IR2: ");
Serial.print(ir2State);
Serial.print(" | Dist: ");
Serial.print(currentDistance);
Serial.print(" cm | Speed: ");
Serial.print(speed);
Serial.println(" cm/s");


// 5. Instant LED Control Logic (ON only when Night AND Motion, OFF immediately when motion is
removed) if (isNightMode) {
// ================= NIGHT MODE =================
if (motionDetected) { controlAllLEDs(HIGH); // Motion
present -> Turn ON instantly
} else { controlAllLEDs(LOW); // Motion removed -> Turn OFF
instantly
}
} else {
// ================= DAYTIME MODE =================
// During daytime, keep all LEDs strictly OFF regardless of motion
controlAllLEDs(LOW);
```

```
}
delay(200); // Balanced delay for stability
}

// Helper function to control all LEDs together
void controlAllLEDs(int state) {
digitalWrite(greenLED, state);
digitalWrite(yellowLED, state);
digitalWrite(redLED, state);
}
```

LumiPath | BCSE316L Design of Smart City | Page 16